

Manual Técnico y de Arquitectura — Agro360

Manual Técnico y de Arquitectura — Agro360

Sistema de Caracterización Predial Agropecuaria Versión: 1.0 — Entrega formal
COA Fecha: Abril 2026

1. Resumen ejecutivo

Agro360 es una aplicación web progresiva (**PWA**) con arquitectura **offline-first** construida sobre **Next.js 16 (App Router)**, **React 19**, **TypeScript** y **Supabase** como backend como servicio (BaaS). Los datos se capturan en el navegador del asesor (IndexedDB vía Dexie) y se sincronizan contra PostgreSQL gestionado por Supabase cuando existe conexión.

El frontend se despliega en **Vercel** (edge network + serverless functions). No hay servidor Node persistente — todo se ejecuta como funciones serverless en demanda.

2. Pila tecnológica (stack)

2.1 Frontend

Componente	Versión	Rol
Next.js	16.0.10	Framework React con App Router, SSR, ISR
React	19.2.0	Biblioteca de UI
TypeScript	5.x	Tipado estático
Tailwind CSS	4.1.9	Sistema de estilos utility-first
Radix UI	1.x	Primitivas accesibles (diálogos, menús, etc.)
shadcn/ui	—	Sistema de componentes basado en Radix + Tailwind
React Hook Form	7.60	Gestión de formularios
Zod	3.25	Validación de esquemas

Componente	Versión	Rol
Dexie	4.3	Wrapper sobre IndexedDB
Leaflet	1.9.4	Mapas interactivos
qrcode.react	4.2	Generación de códigos QR

2.2 Backend / infraestructura

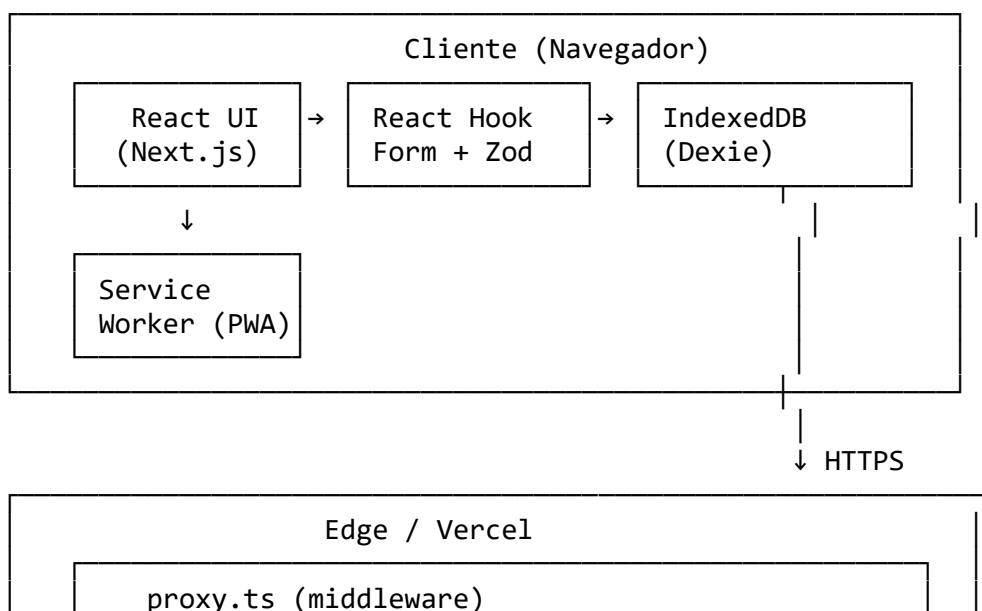
Componente	Rol
Supabase	Auth + PostgreSQL + Storage + RLS
Vercel	Hosting, edge functions, CDN
Nodemailer	Envío de correos (SMTP)
Cloudflare Turnstile	Captcha en formulario público
OpenWeather / Agromonitoring	Datos climáticos y NDVI satelital

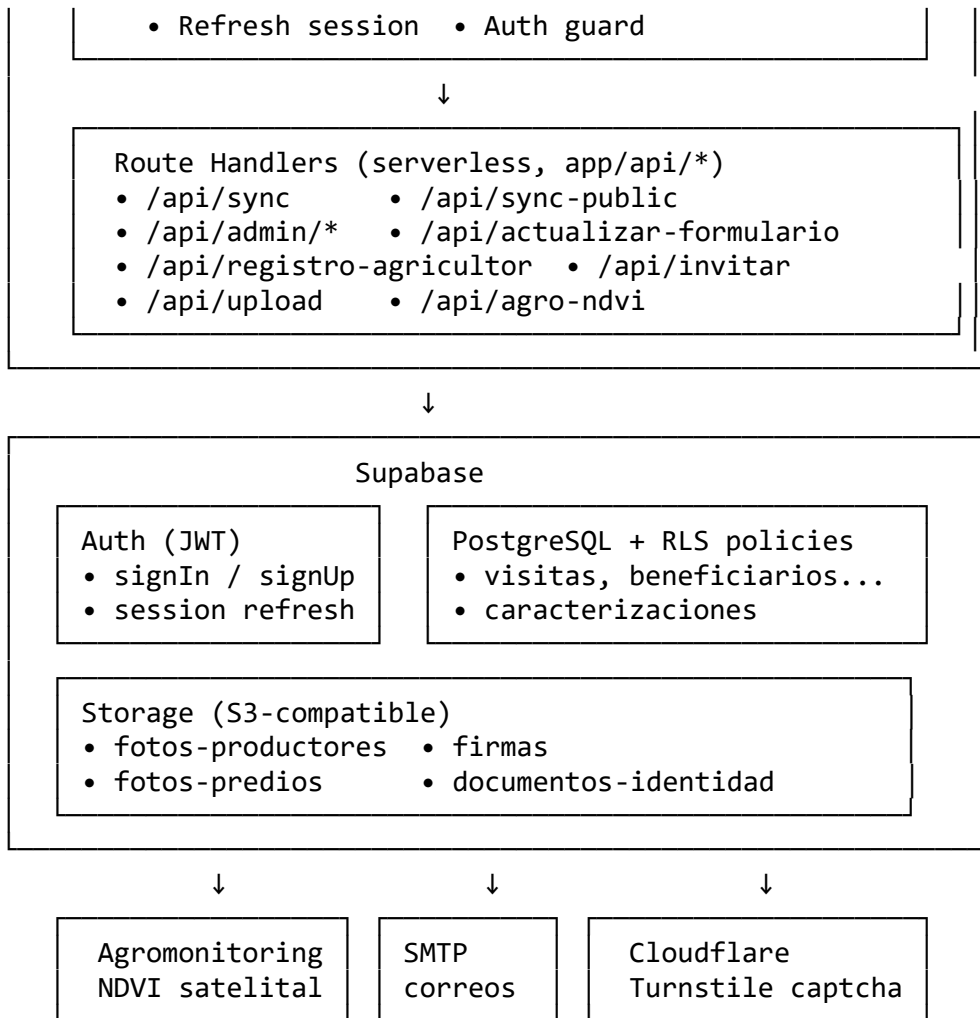
2.3 Base de datos

- **PostgreSQL 15+** gestionado por Supabase.
- **Row Level Security (RLS)** habilitado en todas las tablas.
- Tablas principales: visitas, beneficiarios, predios, caracterizacion_predio, abastecimiento_agua, riesgos_predio, area_productiva, informacion_financiera, caracterizaciones, profiles, invitations.

Ver **Diccionario de Datos** (08-diccionario-datos.md) para esquema completo.

3. Arquitectura de alto nivel





4. Estructura del proyecto

agro-360/

```
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # Layout raíz
│   ├── page.tsx                       # Home
│   ├── globals.css                    # Estilos globales
│   └── (rutas públicas)
│       ├── formulario/                # Formulario de caracterización
│       ├── registro/                  # Registro agricultor
│       ├── auth/                      # Login, signup, forgot-password
│       ├── exito/                     # Confirmación
│       └── offline/                   # Página offline
│   └── (rutas protegidas)
│       ├── dashboard/                 # Dashboard asesor/admin
│       ├── admin/                     # Panel admin
│       └── mapa/                      # Visor NDVI
```

- profile/ # Perfil usuario
 - settings/ # Configuración
 - api/ # Route Handlers (serverless)
 - sync/ # Sync autenticado
 - sync-public/ # Sync sin login (legacy)
 - admin/ # Endpoints admin
 - actualizar-formulario/
 - registro-agricultor/
 - invitar/ # Invitaciones por correo
 - upload/ # Subida archivos
 - beneficiario/
 - caracterizacion/[id]/
 - estado/ # Cambio de estado
 - agro-ndvi/ # NDVI stats
 - agro-tile/ # NDVI tiles
 - agro-images/ # NDVI imagery
 - ndvi/
 - polygon/
 - weather/
 - sentinel-tile/
 - health/ # Health check
- components/
 - characterization-form-complete.tsx # Formulario 9 pasos
- (principal)
 - admin-dashboard.tsx # Dashboard admin
 - agricultor-dashboard.tsx # Dashboard agricultor
 - sync-button.tsx # Botón de sync
 - photo-upload.tsx # Captura de fotos
 - signature-pad.tsx # Firma digital
 - location-picker.tsx # Selector de ubicación
 - map-viewer.tsx # Visor de mapas
 - predio-insights.tsx # Panel NDVI
 - connection-status.tsx # Indicador online/offline
 - sync-error-display.tsx
 - session-validator.tsx
 - user-profile.tsx
 - pwa-provider.tsx
 - theme-provider.tsx
 - theme-toggle.tsx
 - env-var-checker.tsx
 - form-field.tsx
 - ui/ # shadcn/ui components
- context/
 - auth-context.tsx # Provider de autenticación
- hooks/
 - use-auth.ts # Re-exporta AuthContext
 - use-auto-sync.ts # Sync automático en background
 - use-online-status.ts # Detección online/offline
 - use-session-validation.ts
 - use-sync.ts # Lógica de sync

—	use-mobile.ts	
—	use-toast.ts	
—	lib/	
—	db/	
—	indexed-db.ts	# Dexie schema + helpers
—	email/	# Templates HTML correos
—	supabase/	
—	client.ts	# Cliente browser
—	server.ts	# Cliente server (RSC, API)
—	middleware.ts	# updateSession
—	storage.ts	# Helpers Storage
—	generate-pdf.ts	
—	store.ts	
—	utils.ts	# cn() y utilitarios
—	supabase/	
—	migrations/	# Migraciones SQL
—	scripts/	# SQL inicial (legacy)
—	public/	
—	manifest.json	# Manifest PWA
—	sw.js	# Service Worker
—	icons/	# Iconos PWA
—	styles/	
—	types/	
—	leaflet.d.ts	# Declaraciones ambient
—	proxy.ts	# Middleware Next.js 16
—	next.config.mjs	
—	tsconfig.json	
—	package.json	
—	README.md	

5. Patrones arquitectónicos clave

5.1 Offline-first con IndexedDB

El formulario guarda cada entrada en **IndexedDB** (vía Dexie) **antes** de intentar sincronizar. Esto permite:

- Trabajar sin conexión total.
- Recuperarse de caídas de red sin pérdida de datos.
- Retomar un formulario tras cerrar el navegador.

Esquema local (lib/db/indexed-db.ts):

```
class Agro360DB extends Dexie {
  caracterizaciones: EntityTable<CaracterizacionLocal, 'id'>
  backups: EntityTable<BackupLocal, 'id'>
  syncLogs: EntityTable<SyncLog, 'id'>
}
```

Cada CaracterizacionLocal tiene un campo estado: - PENDIENTE_SINCRONIZACION → no enviada - SINCRONIZADO → ya está en servidor - ERROR_SINCRONIZACION → falló el último intento

5.2 Sincronización con el servidor

hooks/use-sync.ts dispara POST /api/sync (asesor autenticado) enviando el array de caracterizaciones pendientes.

El endpoint /api/sync (servidor): 1. Valida JWT del asesor. 2. Para cada caracterización, inserta en cascada: - visitas - beneficiarios (busca por numero_documento; crea si no existe) - predios - caracterizacion_predio, abastecimiento_agua, riesgos_predio, area_productiva - informacion_financiera - Sube fotos/firmas a Supabase Storage y guarda las URLs en las columnas correspondientes de caracterizaciones - caracterizaciones (tabla central que une todo) 3. Genera radicado_oficial (RAD-000XXX). 4. Si el beneficiario tiene correo: crea cuenta Supabase con rol agricultor (si no existe) y envía correo con credenciales o notificación. 5. Retorna { radicadoLocal, radicadoOficial, estado: 'SINCRONIZADO' } por cada caracterización.

El cliente actualiza IndexedDB con los resultados.

5.3 Autenticación

- **Supabase Auth** (JWT en cookies HttpOnly).
- proxy.ts (middleware Next.js 16): refresca sesión en cada request, redirige a /auth/login si ruta protegida y no hay usuario.
- context/auth-context.tsx: provider cliente que expone user, profile, isAsesor, isAdmin, signOut().
- Listener visibilitychange refresca sesión al volver a la pestaña.
- Manejo de refresh_token_not_found → cierre limpio de sesión.

5.4 Row Level Security (RLS)

Todas las tablas tienen RLS habilitado. Políticas principales:

- **Asesor:** SELECT/INSERT/UPDATE sobre sus propias visitas y cascada.
- **Admin:** acceso total.
- **Agricultor:** SELECT sobre sus propias caracterizaciones (vía numero_documento en profile).
- **Analista:** SELECT global, UPDATE de estado en caracterizaciones.

Detalle en scripts/003_complete_agrosantander_schema.sql y supabase/migrations/20260302_update_policies.sql.

5.5 Route Handlers (serverless)

Todos los endpoints en `app/api/*` se ejecutan como **Vercel Functions** (serverless, Node.js 20). Sin estado entre invocaciones.

Endpoints sensibles (admin) usan `SUPABASE_SERVICE_ROLE_KEY` solo en el servidor — nunca se expone al cliente.

5.6 PWA

- `public/manifest.json` declara iconos, colores, scope.
 - `public/sw.js` cachea assets estáticos.
 - `components/pwa-provider.tsx` registra el service worker.
 - Instalable en Android, iOS (con limitaciones), escritorio.
-

6. Seguridad

6.1 Protección de rutas

Doble capa: 1. **Servidor** (`middleware proxy.ts`): redirige si no hay JWT válido. 2. **Cliente** (`AuthContext + redirect`): verifica rol antes de renderizar.

6.2 Claves y secretos

- Variables `NEXT_PUBLIC_*` se exponen al cliente — solo para claves públicas.
- `SUPABASE_SERVICE_ROLE_KEY` solo en servidor — **bypasea RLS, no exponer**.
- SMTP, API keys externas — solo en servidor.

6.3 Captcha

Formulario público (sin login) requiere Cloudflare Turnstile. Validación server-side en `/api/sync-public`.

6.4 Sanitización

- **Zod** valida formas en cliente y servidor.
- **Supabase client** parametriza queries (protección SQL injection automática).
- **React** escapa contenido por defecto (protección XSS).

6.5 HTTPS

Vercel fuerza HTTPS en todos los entornos. Los cookies de sesión son Secure + HttpOnly + SameSite=Lax.

6.6 Cache-Control

Rutas protegidas (/admin, /dashboard, /mapa, /profile, /settings) llevan Cache-Control: no-store para evitar caché en bfcache del navegador (ver next.config.mjs).

7. Flujos críticos

7.1 Flujo de caracterización (asesor autenticado)

1. Asesor abre /formulario
2. Completa paso 1..9 → datos en estado React + IndexedDB (autosave)
3. Al llegar al paso 9 + "Enviar":
 - hook useSync() → POST /api/sync
 - servidor procesa, retorna radicadoOficial
 - cliente actualiza IndexedDB (estado = SINCRONIZADO)
4. Mensaje de éxito y redirección a /dashboard

7.2 Flujo de caracterización (agricultor sin login)

1. Agricultor abre /formulario
2. Completa el formulario (no se auto-llena nombre técnico)
3. Llena Captcha Turnstile en paso 9
4. Envía → POST /api/sync-public (valida captcha)
5. Servidor procesa como si fuera sync pero con asesor_id = null
6. Si proporcionó correo, recibe credenciales de acceso

7.3 Flujo de cambio de estado

Admin/Analista/Asesor abre /dashboard/caracterizacion/[id]
→ Botón "Cambiar estado" → selecciona estado
→ POST /api/admin/update-estado { id, nuevoEstado }
→ Servidor valida transición según rol
→ UPDATE caracterizaciones SET estado = ...
→ (opcional) envío de correo al beneficiario

7.4 Flujo de invitación

Admin en /admin/usuarios → "Invitar" → ingresa email + rol
→ POST /api/invitar
→ Servidor genera token, inserta en invitations
→ Envía correo con enlace /auth/invitation?token=...
→ Usuario clic → /auth/invitation valida token
→ Crea cuenta en auth.users con rol
→ Redirige a login

8. NDVI y datos satelitales

El visor de mapas integra:

- **OpenStreetMap** como base (Leaflet).
- **Agromonitoring API** para NDVI satelital (Sentinel-2):
 - POST /polygons (proxy en /api/polygon): crear polígono.
 - GET /ndvi/history (proxy en /api/agro-ndvi): estadísticas.
 - GET /image/sentinel (proxy en /api/agro-images): tiles satelitales.
- **OpenWeather** para temperatura y precipitación por coordenadas (/api/weather).
- Cache en /api/agro-tile/[...path] para servir tiles sin exponer la API key.

El polígono de Agromonitoring se conserva en estado React (agroPolyId) durante la sesión del visor; si falla la creación (ej. duplicado), se reutiliza el ID existente extrayéndolo del mensaje de error.

9. Correos transaccionales

lib/email/ contiene templates HTML para:

- Notificación de sincronización + credenciales (primer registro).
- Confirmación de radicado (cuentas existentes).
- Invitación a registrarse.
- Cambio de estado (opcional).

Motor: Nodemailer + SMTP (configurado vía env vars).

Reintento: si falla el envío, se reintenta una vez tras 3 segundos. Si falla dos veces, se loguea la alerta (se recomienda revisar config SMTP).

10. Observabilidad

- **Logs** en Vercel Dashboard (retención 1 día plan Hobby, 7 días Pro).
- **Prefijos** para filtrar: [v0], [Sync], [v0 AUTH].
- **Endpoint /api/health** para monitoreo externo.
- **/status** (página interna admin) muestra estado de servicios.

Recomendación post-entrega: integrar Sentry o similar para rastreo de errores en producción.

11. Performance

- **SSG** (static generation) en páginas no autenticadas (`/`, `/auth/*`, `/offline`).
 - **SSR on-demand** en páginas dinámicas (`/dashboard/caracterizacion/[id]`).
 - **Bundle splitting** automático de Next.js.
 - **Leaflet** cargado dinámicamente solo en `/mapa` y donde se necesita.
 - **Imágenes** servidas como `unoptimized: true` (Vercel no facturó optimización adicional).
-

12. Limitaciones conocidas

1. **Cuota IndexedDB**: el navegador puede evictar datos si el dispositivo está lleno. Se recomienda sincronizar frecuentemente.
 2. **iOS Safari PWA**: instalación limitada (sin full offline en algunos casos).
 3. **Firma digital**: requiere pantalla táctil o mouse — no usable solo con teclado.
 4. **Subida de fotos**: imágenes `>10MB` se comprimen automáticamente a calidad `0.8 / max 1600px`.
 5. **Correos SMTP**: sin reintentos persistentes — si el servidor SMTP está caído `>3s`, el correo no se envía.
 6. **NDVI**: requiere API key de Agromonitoring (con plan pago para uso extensivo).
-

13. Extensibilidad

Para agregar nuevos campos al formulario:

1. Agregar columna en Supabase (migración SQL).
2. Actualizar tipo `CaracterizacionLocal` en `lib/db/indexed-db.ts`.
3. Incrementar `version` en Dexie (si cambia schema local).
4. Agregar campo en formulario (`components/characterization-form-complete.tsx`).
5. Actualizar mapeo en `/api/sync/route.ts`.
6. Actualizar vista detalle y exportaciones (`admin-dashboard.tsx`).

Para agregar un nuevo rol:

1. Actualizar `CHECK` constraint en `profiles.rol` (migración SQL).
 2. Actualizar `AuthContext` helpers (`is<NuevoRol>`).
 3. Actualizar políticas RLS.
 4. Actualizar UI condicional.
-

14. Referencias

- **Next.js:** <https://nextjs.org/docs>
- **Supabase:** <https://supabase.com/docs>
- **Dexie:** <https://dexie.org/docs>
- **Radix UI:** <https://www.radix-ui.com/primitives/docs>
- **Leaflet:** <https://leafletjs.com/reference.html>
- **Agromonitoring:** <https://agromonitoring.com/api>

Ver también: - docs/entrega-coa/06-guia-instalacion-despliegue.md —
instalación paso a paso. - docs/entrega-coa/08-diccionario-datos.md — esquema
BD completo.